

SOEN 363 - S
Data Systems for Software Engineers

Individual Assignment 2

TrailLink Rentals

Kaila Quimson - 40240746

Due 14 November 2026

1. Task 1: Data Handling..... 2
1.1 Load the three files..... 3
1.2 First five rows..... 3
1.3 Column names, non-null counts, and data types..... 3
1. Task 2: Data Cleaning..... 4
2.0 Working copies and audit log..... 4
2.1 Remove rows with missing values..... 5
2.2 Remove exact duplicate rows..... 5
2.3 Validate numeric values..... 5
2.4 Parse rental dates..... 6
2.5 Remove orphan foreign keys..... 7
2. Task 3: Queries..... 8
Q1. Equipment with a daily rate greater than CAD 40..... 8
Q2. Filters members on city == "Montreal"..... 8
Q3. Total number of members..... 8
Q4. Rentals with quantity greater than 5..... 9
Q5. Average daily rate across all equipment..... 9
Q6. Equipment with the highest daily rate..... 9
Q7. Equipment count per category..... 9
Q8. Rentals starting strictly after September 15, 2026..... 10
Q9. Total revenue for rentals longer than 3 days..... 10
Q10. Halifax members with at least 4 units in one rental..... 10
Q11. Montreal members with a rental starting in September 2026..... 10
Q12. Ottawa members with more than two rentals..... 11
Q13. Top three equipment by total units rented..... 11
Q14. Member(s) with the highest total spending..... 11
Q15. City or cities with the most rental transactions..... 12
Q16. Camping equipment that has never been rented..... 12
Q17. Rental transactions per month..... 12
Q18. Members with at least one high-value rental..... 13
Q19. Highest-spending member(s) in each city..... 13
Q20. Member-equipment pairs rented more than once..... 14

1. Task 1: Data Handling

The three source files come from different systems and formats. Each was loaded into its own Pandas DataFrame and inspected before cleaning.

1.1 Load the three files

```python
# equipment comes from a CSV file
equipment_raw = pd.read_csv("Tables/equipment.csv")

# members comes from an Excel file (needs openpyxl)
members_raw = pd.read_excel("Tables/members.xlsx", engine="openpyxl")

# rentals is a .txt file but it's comma-separated, so read_csv works
rentals_raw = pd.read_csv("Tables/rentals.txt", sep=",")

# quick summary to check everything loaded
# each key ("table", "file", ...) becomes a column header; each list fills that column
summary = pd.DataFrame({
    "table": ["equipment", "members", "rentals"],
    "file": ["equipment.csv", "members.xlsx", "rentals.txt"],
    "rows": [len(equipment_raw), len(members_raw), len(rentals_raw)],
    "columns": [equipment_raw.shape[1], members_raw.shape[1], rentals_raw.shape[1]],
})
show(summary)
```

✓ 0.1s Python

table	file	rows	columns
equipment	equipment.csv	17	5
members	members.xlsx	14	4
rentals	rentals.txt	67	6

1.2 First five rows

```python
print("Equipment")
show(equipment_raw.head())

print("Members")
show(members_raw.head())

print("Rentals")
show(rentals_raw.head())
```

✓ 0.0s Python

```
Equipment
```

equipment id	equipment name	category	daily rate	stock quantity
1	City Bike	Cycling	25	12
2	Mountain Bike	Cycling	45	8
3	Road Bike	Cycling	65	5
4	Kayak	Water Sports	55	6
5	Paddleboard	Water Sports	40	7

```
Members
```

member id	member name	email	city
1	Alex Roy	member1@example.com	Montreal
2	Sam Chen	member2@example.com	Ottawa
3	Maya Patel	member3@example.com	Halifax
4	Noah Martin	member4@example.com	Montreal
5	Lina Ali	member5@example.com	Ottawa

```
Rentals
```

rental id	member id	equipment id	quantity	rental days	rental date
1	3	1	5.0	2	2026-09-03
2	2	9	6.0	4	2026-08-25
3	7	3	6.0	2	2026-09-07
4	1	2	2.0	5	2026-09-05
5	11	11	2.0	5	2026-10-11

1.3 Column names, non-null counts, and data types

```
# same info as .info(), but as a table: column, type, non-null and missing counts
def column_info(df):
    return pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "non-null": df.notna().sum().values,
        "missing": df.isna().sum().values,
    })

print("Equipment")
show(column_info(equipment_raw))

print("Members")
show(column_info(members_raw))

print("Rentals")
show(column_info(rentals_raw))
```

✓ 0.0s

Equipment

column	dtype	non-null	missing
equipment_id	int64	17	0
equipment_name	str	17	0
category	str	16	1
daily_rate	int64	17	0
stock_quantity	int64	17	0

Members

column	dtype	non-null	missing
member_id	int64	14	0
member_name	str	14	0
email	str	13	1
city	str	14	0

Rentals

column	dtype	non-null	missing
rental_id	int64	67	0
member_id	int64	67	0
equipment_id	int64	67	0
quantity	float64	66	1
rental_days	int64	67	0
rental_date	str	67	0

2. Task 2: Data Cleaning

Cleaning was done on copies of the raw DataFrames, so the original files stay unchanged. The required rules were applied in order, and each step logs the rows removed from every table, including zero removals. A row removed in one step is never countered again later.

2.0 Working copies and audit log

```
# work on copies so the original data stays untouched
equipment = equipment_raw.copy()
members = members_raw.copy()
rentals = rentals_raw.copy()

# audit log: one row per table per step
audit = []

# record how many rows a step removed
def log(table, step, before, after):
    audit.append({"table": table, "step": step, "before": before,
                  "removed": before - after, "after": after})

# starting row counts
log("equipment", "0. Initial", len(equipment), len(equipment))
log("members", "0. Initial", len(members), len(members))
log("rentals", "0. Initial", len(rentals), len(rentals))

# show the starting counts
show(pd.DataFrame(audit))
```

✓ 0.0s Python

table	step	before	removed	after
equipment	0. Initial	17	0	17
members	0. Initial	14	0	14
rentals	0. Initial	67	0	67

2.1 Remove rows with missing values

All columns are required, so any row with an empty value is removed.

```python
# equipment: drop rows with any empty cell
before = len(equipment)
equipment = equipment.dropna()
log("equipment", "1. Missing values", before, len(equipment))

# members: same
before = len(members)
members = members.dropna()
log("members", "1. Missing values", before, len(members))

# rentals: same
before = len(rentals)
rentals = rentals.dropna()
log("rentals", "1. Missing values", before, len(rentals))

# show this step's results: last 3 rows of the audit (step name is in the heading above)
show(pd.DataFrame(audit).tail(3).drop(columns="step"))
```

0.0s Python

table	before	removed	after
equipment	17	1	16
members	14	1	13
rentals	67	1	66

2.2 Remove exact duplicate rows

Duplicates are checked across all columns, keeping the first copy. Repeat rentals of the same member and equipment are valid and are not removed.

```python
# equipment: drop rows that are exact copies, keep the first one
before = len(equipment)
equipment = equipment.drop_duplicates(keep="first")
log("equipment", "2. Exact duplicates", before, len(equipment))

# members: same
before = len(members)
members = members.drop_duplicates(keep="first")
log("members", "2. Exact duplicates", before, len(members))

# rentals: compares all columns, so repeat rentals (same member + equipment) stay
before = len(rentals)
rentals = rentals.drop_duplicates(keep="first")
log("rentals", "2. Exact duplicates", before, len(rentals))

# show this step's results: last 3 rows of the audit (step name is in the heading above)
show(pd.DataFrame(audit).tail(3).drop(columns="step"))
```

0.0s Python

table	before	removed	after
equipment	16	1	15
members	13	1	12
rentals	66	1	65

2.3 Validate numeric values

Numeric fields were converted with pd.to_numeric(errors="coerce"), which turns invalid values into missing values.

Rules:

- All IDs, quantity, and rental_days must be positive integers
- daily_rate must be at least CAD 1.00
- Stock_quantity must be a non-negative integer (zero stock is valid)

Rows that fail any of these rules are removed.

```python
# True when the value exists and is a whole number (5.0 counts as 5)
def is_whole(col):
    return col.notna() & (col == col.round())

# --- equipment ---
before = len(equipment)

# convert to numbers; anything invalid becomes missing
equipment["equipment_id"] = pd.to_numeric(equipment["equipment_id"], errors="coerce")
equipment["daily_rate"] = pd.to_numeric(equipment["daily_rate"], errors="coerce")
equipment["stock_quantity"] = pd.to_numeric(equipment["stock_quantity"], errors="coerce")

# rules: positive whole ID, rate at least 1.00, stock 0 or more
valid = (
    is_whole(equipment["equipment_id"]) & (equipment["equipment_id"] > 0)
    & (equipment["daily_rate"] >= 1.00)
    & is_whole(equipment["stock_quantity"]) & (equipment["stock_quantity"] >= 0)
)
equipment = equipment[valid]   # keep only valid rows

# store whole numbers as int, money as float
equipment = equipment.astype({"equipment_id": int, "daily_rate": float, "stock_quantity": int})
log("equipment", "3. Numeric checks", before, len(equipment))

# --- members ---
before = len(members)

# convert ID to a number
members["member_id"] = pd.to_numeric(members["member_id"], errors="coerce")

# rule: positive whole ID
valid = is_whole(members["member_id"]) & (members["member_id"] > 0)
members = members[valid]   # keep only valid rows
members = members.astype({"member_id": int})
log("members", "3. Numeric checks", before, len(members))

# --- rentals ---
before = len(rentals)

# convert all number columns
rentals["rental_id"] = pd.to_numeric(rentals["rental_id"], errors="coerce")
rentals["member_id"] = pd.to_numeric(rentals["member_id"], errors="coerce")
rentals["equipment_id"] = pd.to_numeric(rentals["equipment_id"], errors="coerce")
rentals["quantity"] = pd.to_numeric(rentals["quantity"], errors="coerce")
rentals["rental_days"] = pd.to_numeric(rentals["rental_days"], errors="coerce")

# rule: every one must be a positive whole number
valid = (
    is_whole(rentals["rental_id"]) & (rentals["rental_id"] > 0)
    & is_whole(rentals["member_id"]) & (rentals["member_id"] > 0)
    & is_whole(rentals["equipment_id"]) & (rentals["equipment_id"] > 0)
    & is_whole(rentals["quantity"]) & (rentals["quantity"] > 0)
    & is_whole(rentals["rental_days"]) & (rentals["rental_days"] > 0)
)
rentals = rentals[valid]   # keep only valid rows

# quantity was float (5.0), now back to int
rentals = rentals.astype({"rental_id": int, "member_id": int, "equipment_id": int,
                          "quantity": int, "rental_days": int})
log("rentals", "3. Numeric checks", before, len(rentals))

# show this step's results: last 3 rows of the audit (step name is in the heading above)
show(pd.DataFrame(audit).tail(3).drop(columns="step"))
```

✓ 0.0s Python

table	before	removed	after
equipment	15	2	13
members	12	0	12
rentals	65	2	63

2.4 Parse rental dates

rental_date was parsed with the YYYY-MM-DD format and errors="coerce," so impossible dates (such as 2026-02-30) become missing and are removed. Only rentals have a date column, so the other tables show 0 removals.

```python
# no date column in equipment or members, so log 0
log("equipment", "4. Invalid dates (n/a)", len(equipment), len(equipment))
log("members", "4. Invalid dates (n/a)", len(members), len(members))

# rentals: parse YYYY-MM-DD; impossible dates (like Feb 30) become missing
before = len(rentals)
rentals["rental_date"] = pd.to_datetime(rentals["rental_date"], format="%Y-%m-%d", errors="coerce")
rentals = rentals.dropna(subset=["rental_date"])   # drop the bad dates
log("rentals", "4. Invalid dates", before, len(rentals))

# show this step's results: last 3 rows of the audit (step name is in the heading above)
show(pd.DataFrame(audit).tail(3).drop(columns="step"))
```

✓ 0.0s Python

table	before	removed	after
equipment	13	0	13
members	12	0	12
rentals	63	1	62

2.5 Remove orphan foreign keys

Keep a rental only if its member_id exists in the cleaned members table and its equipment_id exists in the cleaned equipment table. Equipment and members have no foreign keys, so they show 0 removals.

```python
# parent tables have no foreign keys, so log 0
log("equipment", "5. Orphan foreign keys (n/a)", len(equipment), len(equipment))
log("members", "5. Orphan foreign keys (n/a)", len(members), len(members))

# keep a rental only if its member AND equipment exist in the cleaned tables
before = len(rentals)
rentals = rentals[
    rentals["member_id"].isin(members["member_id"])
    & rentals["equipment_id"].isin(equipment["equipment_id"])
]
log("rentals", "5. Orphan foreign keys", before, len(rentals))

# show this step's results: last 3 rows of the audit (step name is in the heading above)
show(pd.DataFrame(audit).tail(3).drop(columns="step"))
```

✓ 0.0s Python

table	before	removed	after
equipment	13	0	13
members	12	0	12
rentals	62	2	60

2.6 Final counts, primary key check, and full audit

```python
# full audit table, grouped by table then step
audit_df = pd.DataFrame(audit).sort_values(["table", "step"])
show(audit_df)

# final summary: row counts + primary key check (True = no repeated IDs)
final_check = pd.DataFrame({
    "table": ["equipment", "members", "rentals"],
    "final rows": [len(equipment), len(members), len(rentals)],
    "primary key": ["equipment_id", "member_id", "rental_id"],
    "PK unique": [equipment["equipment_id"]. is_unique, members["member_id"].is_unique, rentals["rental_id"].is_unique],
})
show(final_check)
```

✓ 0.0s Python

table	step	before	removed	after
equipment	0. Initial	17	0	17
equipment	1. Missing values	17	1	16
equipment	2. Exact duplicates	16	1	15
equipment	3. Numeric checks	15	2	13
equipment	4. Invalid dates (n/a)	13	0	13
equipment	5. Orphan foreign keys (n/a)	13	0	13
members	0. Initial	14	0	14
members	1. Missing values	14	1	13
members	2. Exact duplicates	13	1	12
members	3. Numeric checks	12	0	12
members	4. Invalid dates (n/a)	12	0	12
members	5. Orphan foreign keys (n/a)	12	0	12
rentals	0. Initial	67	0	67
rentals	1. Missing values	67	1	66
rentals	2. Exact duplicates	66	1	65
rentals	3. Numeric checks	65	2	63
rentals	4. Invalid dates	63	1	62
rentals	5. Orphan foreign keys	62	2	60

table	final rows	primary key	PK unique
equipment	13	equipment_id	True
members	12	member_id	True
rentals	60	rental_id	True

Effect on joins: After cleaning, every ID appears only once, so when rentals are joined with members and equipment, each rental lines up with exactly one member and one item, and nothing gets counted twice. Rentals that pointed to members or equipment that don't exist were removed, so no rental ends up with blank info or gets quietly dropped in the join. With the invalid numbers and dates gone, the revenue and date queries in Task 3 also give correct results.

3. Task 3: Queries

All queries use the cleaned data only. Revenue = quantity × rental_days × daily_rate. Calculations keep full precision, and money is rounded to two decimals only at the end. "Highest" questions include every tie. Ranked results are sorted by the measure (descending), then by ID; all other results are sorted by ID, or alphabetically when there's no ID.

Q1. Equipment with a daily rate greater than CAD 40

The cleaned rentals are joined with members and equipment once, and a revenue column is added. Most questions reuse this table (df).

```python
# equipment with a daily rate over 40
q1 = equipment[equipment["daily_rate"] > 40]

# keep the id and name, sorted by id
q1 = q1[["equipment_id", "equipment_name"]].sort_values("equipment_id")
show(q1)
```

✓ 0.0s Python

equipment id	equipment name
2	Mountain Bike
3	Road Bike
4	Kayak
12	Expedition Tent

Q2. Filters members on city == "Montreal"

Members who live in Montreal. A boolean filter keeps rows where city equals "Montreal", showing ID and name sorted by ID.

```python
# members who live in Montreal
q2 = members[members["city"] == "Montreal"]

# keep the id and name, sorted by id
q2 = q2[["member_id", "member_name"]].sort_values("member_id")
show(q2)
```

✓ 0.0s Python

member id	member name
1	Alex Roy
4	Noah Martin
8	Eva Kim

Q3. Total number of members

Total number of members. len() counts every row in the cleaned members table, including members with no rentals.

```python
# every row in the cleaned members table (including members with no rentals)
q3 = len(members)
print("Total members:", q3)
```

✓ 0.0s Python

```
Total members: 12
```

Q4. Rentals with quantity greater than 5

Rentals with more than 5 units. A filter keeps rentals where quantity > 5, and count() counts their rental IDs.

```python
# rentals with more than 5 units
big_rentals = rentals[rentals["quantity"] > 5]

# count their rental ids
q4 = big_rentals["rental_id"].count()
print("Rentals with quantity > 5:", q4)
```

0.0s Python

```
Rentals with quantity > 5: 14
```

Q5. Average daily rate across all equipment

Average daily rate. mean() averages daily_rate across all cleaned equipment, and the result is rounded to 2 decimals only when printed.

```python
# highest daily rate
max_rate = equipment["daily_rate"].max()

# every item at that rate (keeps ties)
q6 = equipment[equipment["daily_rate"] == max_rate]
q6 = q6[["equipment_id", "equipment_name", "daily_rate"]].sort_values("equipment_id")
show(q6)
```

0.0s Python

equipment id	equipment name	daily rate
12	Expedition Tent	85.0

Q6. Equipment with the highest daily rate

Equipment with the highest daily rate. max() finds the top rate, and then a filter keeps every item equal to it so ties are included.

Q6. Equipment with the highest daily rate

Finds the maximum rate, then keeps every item at that rate so ties are included.


```python
# highest daily rate
max_rate = equipment["daily_rate"].max()

# every item at that rate (keeps ties)
q6 = equipment[equipment["daily_rate"] == max_rate]
q6 = q6[["equipment_id", "equipment_name", "daily_rate"]].sort_values("equipment_id")
show(q6)
```

0.0s Python

equipment id	equipment name	daily rate
12	Expedition Tent	85.0

Q7. Equipment count per category

Equipment count per category. The equipment table is grouped by category, and count() counts the items in each group. Using equipment instead of rentals means items that were never rented are still counted. Results are sorted alphabetically.

```python
# count items per category using the equipment table (so never-rented items count too)
q7 = equipment.groupby("category")["equipment_id"].count().reset_index()
q7.columns = ["category", "equipment_count"]

# no id here, so sort alphabetically
q7 = q7.sort_values("category")
show(q7)
```

✓ 0.0s Python

category	equipment count
Camping	4
Climbing	1
Cycling	4
Hiking	2
Water Sports	2

Q8. Rentals starting strictly after September 15, 2026

Rentals starting strictly after September 15, 2026. A filter keeps rentals with rental_date > 2026-09-15; the strict > excludes September 15 itself. count() gives the total.

```python
# rentals that start strictly after Sept 15, 2026 (Sept 15 itself not included)
after_sep15 = rentals[rentals["rental_date"] > "2026-09-15"]

# count their rental ids
q8 = after_sep15["rental_id"].count()
print("Rentals after 2026-09-15:", q8)
```

✓ 0.0s Python

```
Rentals after 2026-09-15: 27
```

Q9. Total revenue for rentals longer than 3 days

Total revenue for rentals longer than 3 days. The joined table is filtered on rental_days > 3, and sum() adds up the revenue column, rounded only at the end.

```python
# rentals longer than 3 days
long_rentals = df[df["rental_days"] > 3]

# add up their revenue, round only at the end
q9 = long_rentals["revenue"].sum()
print(f"Total revenue (rental_days > 3): CAD {q9:.2f}")
```

✓ 0.0s Python

```
Total revenue (rental_days > 3): CAD 14597.00
```

Q10. Halifax members with at least 4 units in one rental

Halifax members with at least 4 units in one rental. Two conditions are combined with & (city is Halifax and quantity ≥ 4), and drop_duplicates() lists each member once.

```python
# Halifax members with at least 4 units in one rental
q10 = df[(df["city"] == "Halifax") & (df["quantity"] >= 4)]

# one row per member, sorted by id
q10 = q10[["member_id", "member_name"]].drop_duplicates().sort_values("member_id")
show(q10)
```

✓ 0.0s Python

member id	member name
3	Maya Patel
6	Emma Wilson

Q11. Montreal members with a rental starting in September 2026

Montreal members with a rental in September 2026. The filter checks city = Montreal and uses .dt.year and .dt.month to keep September 2026 rentals. drop_duplicates() lists each member once.

```python
# Montreal members with a rental starting in September 2026
sept = df[(df["city"] == "Montreal")
          & (df["rental_date"].dt.year == 2026)
          & (df["rental_date"].dt.month == 9)]

# one row per member, sorted by id
q11 = sept[["member_id", "member_name"]].drop_duplicates().sort_values("member_id")
show(q11)
```

✓ 0.0s Python

member id	member name
1	Alex Roy
4	Noah Martin
8	Eva Kim

Q12. Ottawa members with more than two rentals

Ottawa members with more than two rentals. Ottawa rentals are grouped by member, and count() gives each member's number of rentals; a second filter keeps counts above 2.

```python
# count rentals for each Ottawa member
ottawa = df[df["city"] == "Ottawa"]
q12 = ottawa.groupby(["member_id", "member_name"])["rental_id"].count().reset_index()
q12.columns = ["member_id", "member_name", "rental_count"]

# keep members with more than two rentals
q12 = q12[q12["rental_count"] > 2].sort_values("member_id")
show(q12)
```

✓ 0.0s Python

member id	member name	rental count
2	Sam Chen	6

Q13. Top three equipment by total units rented

Top three equipment by total units rented. groupby() and sum() total the quantity per item (units, not number of rentals). sort_values() orders by total descending and ID ascending to break ties, and head(3) returns exactly three rows

```python
# total units rented per item (sum of quantity, not number of rentals)
q13 = df.groupby(["equipment_id", "equipment_name"])["quantity"].sum().reset_index()
q13.columns = ["equipment_id", "equipment_name", "total_units"]

# most units first, ties broken by lower id, then exactly 3 rows
q13 = q13.sort_values(["total_units", "equipment_id"], ascending=[False, True]).head(3)
show(q13)
```

✓ 0.0s Python

equipment id	equipment name	total units
1	City Bike	32
9	Climbing Kit	31
3	Road Bike	30

Q14. Member(s) with the highest total spending

Member(s) with the highest total spending. groupby() and sum() total the revenue per member, then a filter keeps every member equal to the maximum so ties are included. Money is rounded at the end.

```python
# total spending per member (also reused in Q19)
spending = df.groupby(["member_id", "member_name", "city"])["revenue"].sum().reset_index()
spending.columns = ["member_id", "member_name", "city", "total_spending"]

# member(s) with the highest total (keeps ties)
top = spending["total_spending"].max()
q14 = spending[spending["total_spending"] == top]
q14 = q14[["member_id", "member_name", "total_spending"]].sort_values("member_id")

# round the money only at the end
q14["total_spending"] = q14["total_spending"].round(2)
show(q14)
```

✓ 0.0s Python

member id	member name	total spending
7	Omar Bell	4058.0

Q15. City or cities with the most rental transactions

City or cities with the most rentals. Rentals are grouped by the member's city and counted, then a filter keeps every city equal to the highest count.

```python
# number of rentals per member city
city_counts = df.groupby("city")["rental_id"].count().reset_index()
city_counts.columns = ["city", "rental_count"]

# city or cities with the most rentals (keeps ties)
top = city_counts["rental_count"].max()
q15 = city_counts[city_counts["rental_count"] == top].sort_values("city")
show(q15)
```

✓ 0.0s Python

city	rental count
Montreal	19

Q16. Camping equipment that has never been rented

Camping equipment that has never been rented. Equipment is filtered to the Camping category, then ~isin() keeps only items whose ID never appears in the rentals table (an anti-join)

```python
# Camping equipment
camping = equipment[equipment["category"] == "Camping"]

# keep items whose id never appears in rentals (anti-join)
q16 = camping[~camping["equipment_id"].isin(rentals["equipment_id"])]
q16 = q16[["equipment_id", "equipment_name"]].sort_values("equipment_id")
show(q16)
```

✓ 0.0s Python

equipment id	equipment name
12	Expedition Tent

Q17. Rental transactions per month

Rentals per month. .dt.strftime("%Y-%m") turns each date into its year-month, value_counts() counts rentals per month, and sort_index() puts the months in chronological order.

```python
# turn each date into its year-month, e.g. 2026-09
months = rentals["rental_date"].dt.strftime("%Y-%m")

# count rentals per month, oldest month first
q17 = months.value_counts().sort_index().reset_index()
q17.columns = ["year_month", "rental_count"]
show(q17)
```

✓ 0.0s Python

year month	rental count
2026-08	20
2026-09	20
2026-10	20

Q18. Members with at least one high-value rental

Members with at least one high-value rental. A rental is high value when revenue > 500 OR (quantity ≥ 4 AND the member lives in Montreal). Every rental starts as "Standard"; matching rentals are relabeled with .loc, and the unique members with a high-value rental are listed.

```python
# highest total spending within each city (Q14's table only has members with rentals)
spending["city_max"] = spending.groupby("city")["total_spending"].transform("max")

# keep the member(s) matching their city's max (keeps ties)
q19 = spending[spending["total_spending"] == spending["city_max"]]
q19 = q19[["city", "member_id", "member_name", "total_spending"]].sort_values(["city", "member_id"])

# round the money only at the end
q19["total_spending"] = q19["total_spending"].round(2)
show(q19)
```

Loading...

✓ 0.0s Python

city	member id	member name	total spending
Halifax	6	Emma Wilson	2400.0
Montreal	8	Eva Kim	3266.0
Ottawa	2	Sam Chen	3030.0
Quebec City	7	Omar Bell	4058.0
Toronto	10	Nora Brown	3685.0

Q19. Highest-spending member(s) in each city

Highest-spending member(s) in each city. Q14's per-member spending table (members with rentals only) is reused. groupby("city"). transform("max") gives each row its city's highest spending, and a filter keeps the members that match it, including ties. Sorted by city, then member ID.

```python
# highest total spending within each city (Q14's table only has members with rentals)
spending["city_max"] = spending.groupby("city")["total_spending"].transform("max")

# keep the member(s) matching their city's max (keeps ties)
q19 = spending[spending["total_spending"] == spending["city_max"]]
q19 = q19[["city", "member_id", "member_name", "total_spending"]].sort_values(["city", "member_id"])

# round the money only at the end
q19["total_spending"] = q19["total_spending"].round(2)
show(q19)
```

✓ 0.0s Python

city	member id	member name	total spending
Halifax	6	Emma Wilson	2400.0
Montreal	8	Eva Kim	3266.0
Ottawa	2	Sam Chen	3030.0
Quebec City	7	Omar Bell	4058.0
Toronto	10	Nora Brown	3685.0

Q20. Member-equipment pairs rented more than once

Member-equipment pairs rented more than once. Rentals are grouped by member_id and equipment_id and counted, then a filter keeps pairs with a count above 1.

```python
# count rentals for each member + equipment pair
q20 = rentals.groupby(["member_id", "equipment_id"])["rental_id"].count().reset_index()
q20.columns = ["member_id", "equipment_id", "rental_count"]

# keep pairs rented more than once
q20 = q20[q20["rental_count"] > 1].sort_values(["member_id", "equipment_id"])
show(q20)
```

✓ 0.0s Python

member id	equipment id	rental count
1	2	2
2	5	2
3	1	2
6	2	2
7	3	3
7	7	2
8	4	3
8	6	2
10	4	2